

Lab 2 Writeup — ADT Implementations + Performance Shootout

Author: Dylan Spivey **Course:** COSC 310 — Advanced Java **Date:** April 26, 2026

Setup

- **JDK:** Eclipse Adoptium 21.0.10.7 (HotSpot), Windows 11
- **Benchmark config:** WARMUP_OPS = 15_000, MEASURE_OPS = 60_000, TRIALS = 7 (median reported), fixed seed 315_351_107L, 2 warmup runs per workload, checksum verified equal across all 7 trials
- All sanity checks passed before timing began

Results (median ns/op)

```
== Stack: ArrayListStack ==
  Workload1 bulk push+pop          5.85
  Workload2 mixed steady-state    23.61

== Stack: DLinkedListStack ==
  Workload1 bulk push+pop          12.76
  Workload2 mixed steady-state    80.47

== Queue: ArrayListQueue ==
  Workload1 bulk enq+deq           9.02
  Workload2 mixed steady-state    22.92

== Queue: DLinkedListQueue ==
  Workload1 bulk enq+deq           9.21
  Workload2 mixed steady-state    50.64

== PriorityQueue: SortedArrayListPQ ==
  W1 bulk enq+deq (uniform priorities) 3095.83
  W2 mixed steady-state (uniform)      3185.74
  W3 skewed priorities (bulk)          3029.03

== PriorityQueue: SortedDLinkedListPQ ==
  W1 bulk enq+deq (uniform priorities) 25226.37
  W2 mixed steady-state (uniform)      26067.08
  W3 skewed priorities (bulk)          21073.39

== PriorityQueue: BinaryHeapPQ ==
  W1 bulk enq+deq (uniform priorities)  74.18
  W2 mixed steady-state (uniform)       56.34
  W3 skewed priorities (bulk)           53.21
```

Stack — ArrayList wins (~2x bulk, ~3.4x mixed)

Both push/pop are $O(1)$ amortized for both implementations, so the difference is entirely in **constant factors**, not Big-O.

The ArrayListStack stores values in a contiguous backing array. push and pop touch the slot at size-1, which is almost always already in L1/L2 cache after the previous operation. The CPU's prefetcher loves linear access, branch prediction is trivial, and the backing array's amortized growth (doubling) means most pushes are just an array write plus a size increment.

The `DLinkedListStack` allocates a fresh `DNode` on every push — that's a heap allocation, GC pressure, and a guaranteed cache miss when you later traverse to that node. Each pop chases a prev pointer to a node that may live anywhere in heap memory. This pointer chasing is exactly what kills linked structures on modern CPUs: the hardware can't prefetch what it can't predict, so each operation pays a cache miss penalty (~10–100 ns).

The gap widens dramatically in W2 (80 vs 24 ns/op) because the mixed workload causes more frequent allocation/deallocation cycles in the linked list, which churns the GC and pollutes the cache further.

Queue — ArrayList wins (tied bulk, ~2.2x mixed)

`ArrayListQueue` is implemented as a **circular buffer** — head and tail are integer indices that wrap around modulo capacity. enqueue writes to `buffer[tail]` and bumps tail; dequeue reads `buffer[head]` and bumps head. No shifting, no allocation per op (only when capacity doubles). The hot data lives in a single contiguous array.

`DLinkedListQueue` allocates a `DNode` on every enqueue and frees one on every dequeue. The pointer-chase cost is identical to the stack case.

In W1 (bulk fill, then bulk drain), the two are nearly tied (9.0 vs 9.2). I think this is because both can stream linearly through memory in this pattern — the linked list traversal during drain hits each node exactly once in the order they were allocated, which on a fresh JVM is roughly contiguous in the young generation. So the cache-miss penalty is small.

In W2 (mixed steady-state at ~10k items), the linked list's nodes get scattered across the heap as GC moves them around, and the ArrayList's circular buffer keeps everything packed and predictable — so the gap opens up to 2.2x.

Priority Queue — Binary Heap dominates (40–490x)

This is the biggest gap and the most interesting result.

Big-O recap:

- `SortedListPQ`: enqueue is $O(N)$ (linear scan + array shift), dequeue is $O(N)$ (shift everything left after `remove(0)`). Front is $O(1)$.
- `SortedListPQ`: enqueue is $O(N)$ (linear traversal to find insertion point), dequeue is $O(1)$ (`removeFirst`). Front is $O(1)$.
- `BinaryHeapPQ`: enqueue is $O(\log N)$, dequeue is $O(\log N)$, front is $O(1)$.

For $N = 30,000$ (the bulk workload), $\log_2(30000) \approx 15$ while $N = 30,000$. That's a **2000x asymptotic advantage** for the heap. The observed speedup of 40–490x is a bit smaller because the heap also has overhead (sift-up/sift-down loops, swaps, capacity checks).

Why is the sorted DLinkedList so much worse than the sorted ArrayList (8x), even though both are $O(N)$ for insert? The constant factor. The sorted ArrayList does its $O(N)$ work as a tight loop reading sequential memory, then `System.arraycopy` for the shift — both are heavily optimized by the CPU's prefetcher and SIMD. The sorted DLinkedList's $O(N)$ insert is N pointer-chases through scattered heap addresses, where every `curr.getNext()` is potentially a cache miss. The Big-O is the same, but the wall-clock cost per operation is dominated by memory latency on the linked list.

Why does the heap typically dominate at large N? 1. **Logarithmic vs linear work.** Sift operations only touch $O(\log N)$ array slots — for 30,000 items, that's ~15 comparisons and swaps per op vs ~15,000 for the sorted versions. 2. **Cache-friendly layout.** A binary heap is stored as a flat array with parent/child indices computed via $(i-1)/2$ and $2*i+1/2*i+2$. The path from any node to the root touches $O(\log N)$ slots that are spread across the array, but **each siftDown step accesses three consecutive slots** (parent, left child, right child) — those are usually on the same cache line until you get near the bottom of the heap. 3. **No shifting.** Unlike the sorted ArrayList's `remove(0)` (which has to slide N-1 elements), the heap's removal swaps the root with the last element and sifts down — a constant amount of memory movement plus the log-depth sift.

What patterns can make sorted structures look better, or worse?

The W3 skewed-priorities workload (90% in [0..10], 10% in [11..100000]) actually made the sorted DLinkedList **faster** than W1 (21073 vs 25226 ns/op). My read: with most priorities clustered near the front and equal priorities going *after* existing equal ones (FIFO stability), inserting a low-priority item walks past a long run of equal-priority nodes — but inserting a high-priority item terminates the scan quickly once it hits any of the 90% small values. The mix evens out, and the constant factor improves slightly.

Conversely, sorted structures could look better than expected if:

- **The dataset is small** (say $N < 100$). At small N, the heap's constant overhead per op (capacity checks, swap function calls, two siftDown comparisons) can outweigh the savings, and a sorted ArrayList's tight loop wins.
- **Workload is dequeue-heavy and FIFO-ordered priorities.** A sorted DLinkedList's $O(1)$ `removeFirst` becomes the dominant op; if inserts happen rarely or in already-sorted order, the linked list looks competitive.

Sorted structures look **worse** than usual when:

- **Insertion order is adversarial** (e.g., always inserting the new minimum, forcing a linear scan to the front). The heap remains $O(\log N)$ regardless.
- **The list is large** — the linear cost grows with N while the heap grows only with $\log N$.

Conclusion

Across all three ADTs, the ArrayList-backed implementation outperformed the DLinkedList-backed one due to cache locality, predictable access patterns, and the absence of per-op heap allocation. For the priority queue specifically, the binary heap is in a different complexity class than either sorted implementation, and its advantage compounds with N. The lab's "performance shootout" framing matches the data: the same Big-O can hide an order-of-magnitude difference in real wall-clock cost, and a better Big-O can hide three orders of magnitude.